# Practical 1:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 256
#define MAX_ARGS 32

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    pid_t pid;

    printf("Parent Process Started (PID: %d)\n", getpid());
    printf("Enter a Linux command (e.g., 'ls -l' or 'uname -a'): ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        perror("Error reading input");
        return 1;
    }

    // Remove trailing newline character
    input[strcspn(input, "\n")] = '\0';

    // Tokenize the input string into command and arguments
    int i = 0;
    char *token = strtok(input, " ");
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL; // NULL-terminate the arguments array

    if (args[0] == NULL) {
        printf("No command entered.\n");
        return 0;
    }

    // Create a child process
```

```c
// Create a child process
pid = fork();

if (pid < 0) {
    // Fork failed
    perror("fork failed");
    return 1;
} else if (pid == 0) {
    // Child process
    printf("\n[Child Process] PID: %d, Parent PID: %d\n", getpid(), getppid());
    printf("[Child Process] Executing command: %s\n\n", args[0]);

    // Replace child process image with the specified command
    if (execvp(args[0], args) == -1) {
        perror("execvp failed");
        exit(EXIT_FAILURE);
    }
} else {
    // Parent process
    int status;
    printf("\n[Parent Process] Created child with PID: %d. Waiting for completion...\n", pid);

    // Wait for the child process to terminate
    wait(&status);

    if (WIFEXITED(status)) {
        printf("\n[Parent Process] Child (PID: %d) exited with status: %d\n", pid, WEXITSTATUS(status));
    } else {
        printf("\n[Parent Process] Child terminated abnormally.\n");
    }
}

return 0;
```

# Commands:

```
bhanu@DESKTOP-DE32HQ8:~$ nano shell_demo.c
bhanu@DESKTOP-DE32HQ8:~$ gcc shell_demo.c -o shell_demo
bhanu@DESKTOP-DE32HQ8:~$ ./shell_demo
Parent Process Started (PID: 962)
Enter a Linux command (e.g., 'ls -l' or 'uname -a'): ls -l

[Parent Process] Created child with PID: 963. Waiting for completion...

[Child Process] PID: 963, Parent PID: 962
[Child Process] Executing command: ls

total 136
drwxr-xr-x 2 bhanu bhanu  4096 Aug  8 03:39 Projects
-rw-r--r-- 1 bhanu bhanu  2589 Aug 11 06:31 main.c
-rwxr-xr-x 1 bhanu bhanu 16392 Aug 11 06:31 my_cli
-rwxr-xr-x 1 bhanu bhanu 16488 Aug 22 03:03 producer_consumer
-rw-r--r-- 1 bhanu bhanu  2240 Aug 22 03:03 producer_consumer.c
-rwxr-xr-x 1 bhanu bhanu 16528 Aug 22 03:08 shell_demo
-rw-r--r-- 1 bhanu bhanu  1999 Aug 22 03:08 shell_demo.c
-rwxr-xr-x 1 bhanu bhanu 16312 Aug 22 03:04 shell_pipe
-rw-r--r-- 1 bhanu bhanu  1679 Aug 22 03:04 shell_pipe.c
-rwxr-xr-x 1 bhanu bhanu 16168 Aug  7 09:16 zombie
-rw-r--r-- 1 bhanu bhanu   399 Aug  7 09:16 zombie.c
-rwxr-xr-x 1 bhanu bhanu 16312 Aug  7 09:18 zombie_fix
-rw-r--r-- 1 bhanu bhanu  1901 Aug  7 09:22 zombie_fix.c

[Parent Process] Child (PID: 963) exited with status: 0
```

```
bhanu@DESKTOP-DE32HQ8:~$ uname -a
Linux DESKTOP-DE32HQ8 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
bhanu@DESKTOP-DE32HQ8:~$ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             39 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      16
  On-line CPU(s) list:       0-15
Vendor ID:                   GenuineIntel
  Model name:                13th Gen Intel(R) Core(TM) i5-13450HX
    CPU family:              6
    Model:                   183
    Thread(s) per core:      2
    Core(s) per socket:      8
    Socket(s):               1
    Stepping:                1
    BogoMIPS:                5222.39
    Flags:                   fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdts
                             cp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid
                             sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs i
                             bpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflush
                             opt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnmi umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_
                             clear serialize ibt flush_l1d arch_capabilities
Virtualization features:
  Virtualization:            VT-x
  Hypervisor vendor:         Microsoft
  Virtualization type:       full
Caches (sum of all):
  L1d:                       384 KiB (8 instances)
  L1i:                       256 KiB (8 instances)
  L2:                        10 MiB (8 instances)
  L3:                        20 MiB (1 instance)
NUMA:
  NUMA node(s):              1
  NUMA node0 CPU(s):         0-15
Vulnerabilities:
  Gather data sampling:      Not affected
  Ghostwrite:                Not affected
  Indirect target selection: Not affected
  Itlb multihit:             Not affected
  L1tf:                      Not affected
  Mds:                       Not affected
  Meltdown:                  Not affected
  Mmio stale data:           Not affected
```

```
bhanu@DESKTOP-DE32HQ8:~$ uname -a
Linux DESKTOP-DE32HQ8 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for (int i = 1; i <= NUM_ITEMS; i++) {
        // Wait for an empty slot
        sem_wait(&empty);
        // Acquire mutex lock
        pthread_mutex_lock(&mutex);

        // Produce item and insert into buffer
        buffer[in] = i;
        printf("[Producer] Produced item %d at buffer index %d\n", i, in);
        in = (in + 1) % BUFFER_SIZE;

        // Release mutex lock
        pthread_mutex_unlock(&mutex);
        // Signal that a new item is available
        sem_post(&full);

        sleep(1); // Simulate production time
    }
    pthread_exit(NULL);
}

void* consumer(void* arg) {
    for (int i = 1; i <= NUM_ITEMS; i++) {
```

```
void* consumer(void* arg) {
    for (int i = 1; i <= NUM_ITEMS; i++) {
        // Wait for a full slot
        sem_wait(&full);
        // Acquire mutex lock
        pthread_mutex_lock(&mutex);

        // Remove item from buffer
        int item = buffer[out];
        printf("  [Consumer] Consumed item %d from buffer index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        // Release mutex lock
        pthread_mutex_unlock(&mutex);
        // Signal that a slot is free
        sem_post(&empty);

        sleep(2); // Simulate consumption time
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t prod_thread, cons_thread;

    // Initialize synchronization primitives
    sem_init(&empty, 0, BUFFER_SIZE); // All slots start empty
    sem_init(&full, 0, 0);            // 0 full slots initially
    pthread_mutex_init(&mutex, NULL);

    printf("Starting Producer-Consumer Simulation...\n\n");

    // Create producer and consumer threads
    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    // Wait for both threads to finish
    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    // Clean up synchronization primitives
```

# Commands:

```
bhanu@DESKTOP-DE32HQ8:~$ nano shell_demo.c
bhanu@DESKTOP-DE32HQ8:~$ nano producer_consumer.c
bhanu@DESKTOP-DE32HQ8:~$ nano producer_consumer.b
bhanu@DESKTOP-DE32HQ8:~$ gcc producer_consumer.c -o prod_cons -pthread
bhanu@DESKTOP-DE32HQ8:~$ ./prod_cons
[Consumer] Finished receiving. Total bytes: 102400000 bytes

--- Performance Metrics ---
Data Transferred : 97.66 MB (100000 messages of 1024 bytes)
Time Taken       : 0.051489 seconds
Throughput       : 1896.64 MB/sec
bhanu@DESKTOP-DE32HQ8:~$  nano producer_consumer.b
bhanu@DESKTOP-DE32HQ8:~$
```